Grupo I

1. Considere um sistema baseado numa arquitetura em que o respetivo ISA especifica uma organização de memória do tipo *byte-addressable*. Se uma variável de 64 bits, usada num dado programa, se encontrar armazenada em memória a partir do endereço `0x25A1C`, pode concluir-se que a dimensão mínima do espaço de endereçamento dessa arquitetura é:
   a. 2^6
   b. 2^{18}
   c. 2^{32}
   d. 2^{16}

b)

2. Considere uma arquitetura em que o respetivo ISA especifica uma organização de memória do tipo *word-addressable*, em que a dimensão da *word* é 32 bits. Sendo o espaço de endereçamento do processador 24 bits, a máxima memória que este sistema pode acomodar é:
   a. 2^{24} bytes
   b. 2^{32} bytes
   c. 2^{26} bytes
   d. 2^{24} bits

32 bits = 4 bytes
2^24 * 4 = 2^26

3. Uma arquitetura hipotética baseada num modelo *single cycle* em que os operandos das instruções aritméticas e lógicas podem residir em registos internos ou na memória externa, pode ser classificada como:
   a. uma arquitetura Harvard do tipo "Register-Memory"
   b. uma arquitetura Harvard do tipo "Load-store"
   c. uma arquitetura Von Neumann do tipo "Register-Memory"
   d. uma arquitetura Von Neumann do tipo "Load-store"



a)

4. Numa operação de adição de 2 quantidades codificadas em complemento para 2, se o *carry out* do resultado é "0":
   a. pode concluir-se que o resultado é válido, não sendo necessário fazer qualquer outra verificação
   b. pode concluir-se que o resultado não é válido
   c. não é possível, por si só, tirar conclusões quanto à validade do resultado
   d. deve ser calculado o complemento para 2 do valor obtido para se ter o resultado correto

a)

16-01-2020

5. Considere o código máquina `0x0C0000D4` correspondente a uma instrução `jal` do MIPS armazenada no endereço de memória `0x0000A034`. Após a execução dessa instrução:
   a. `$ra=0x0000A038` `PC=0x000001A8`
   b. `$ra=0x0000A034` `PC=0x000000D4`
   c. `$ra=0x0000A038` `PC=0x00000035`
   d. `$ra=0x0000A038` `PC=0x00000350`

$ra = endereço+4 = 0x0000A038
PC = label = 00D4 * 4 = 0350
d)

6. Nas instruções de salto condicional da arquitetura MIPS, para a obtenção do endereço alvo é usado endereçamento
   a. indireto por registo com deslocamento
   b. imediato
   c. relativo ao PC com deslocamento
   d. pseudo-direto

c)

7. Considere o seguinte segmento de código *assembly*:

```
lui   $4,0x36AC
addi  $4,$4,0x6F80
sw    $4,4($4)
lb    $4,6($4)
```

No final da execução do código anterior num MIPS *little endian*, o valor de `$4` é:

a. `0x000036AC` b. `0xFFFFFF80` c. `0x0000006F` d. `0xFFFFFFAC`

$4 = 0x36AC6F80
esse valor será armazenado em 0x36AC6F84
ler o endereço 0x36AC6F86
$4 = AC -> d)

8. Na arquitetura MIPS, uma instrução de *branch* pode saltar para qualquer endereço múltiplo de 4:

a. na gama $[-2^{17}, 2^{17} - 1]$, relativamente ao endereço da instrução seguinte
b. na gama $[-2^{15}, 2^{15} - 1]$, relativamente ao endereço da instrução de *branch*
c. num segmento de memória contíguo de 2^{28} endereços
d. na gama $[-2^{15}, 2^{15} - 1]$, relativamente ao endereço da instrução seguinte

d)

9. Considere que `$2=0xFFFFFFFE` e `$3=0x00000003`. O conteúdo do registo `LO` após a execução da instrução `mult $2,$3` é:

a. `0x00000003`
b. `0x00000006`
c. `0xFFFFFFFF`
d. `0xFFFFFFFA`

$2 = -2
$3 = 3
lo = -6 -> d)

10. Considere que `$2=0xFFFFFFFE` e `$3=0x00000003`. O conteúdo do registo `HI` após a execução da instrução `div $2,$3` é:

a. `0x00000002`
b. `0xFFFFFFFE`
c. `0xFFFFFFFF`
d. `0x00000003`

-2/3
hi = resto = -2 = b)

11. A instrução virtual "`bge $5,0xA3,target`" da arquitetura MIPS decompõe-se na seguinte sequência ordenada de instruções nativas:

a. `slti $1,$5,0xA3` / `beq $1,$0,target`
b. `slti $1,$5,0xA3` / `bne $1,$0,target`
c. `addi $1,$0,0xA3` / `slt $1,$1,$5`
d. `addi $1,$0,0xA3` / `slt $1,$1,$5`

a)

`beq $1,$0,target`
`bne $1,$0,target`

12. Os endereços mínimo e máximo para os quais uma instrução `jr` da arquitetura MIPS, presente no endereço `0x3843F0EC`, pode saltar são, respetivamente:

a. `0x38430000`, `0x3843FFFC`
b. `0x30000000`, `0x3FFFFFFC`
c. `0x0843F0EC`, `0xF843F0EC`
d. `0x00000000`, `0xFFFFFFFC`

b) ?

13. Os processadores da arquitetura hipotética A20 implementam um total de 130 instruções, todas codificadas com 30 bits. Um dos formatos de codificação tem 4 campos: *opcode*, dois campos para identificar registos internos e um campo para codificar valores imediatos na gama [-32768, 32767]. Pode então concluir-se que o número de registos internos dessa arquitetura é:

a. 16
b. 8
c. 32
d. 4

130 <= 256 = 2^8
log2(32768*2+1) = 16
30 = 8 + 2*x + 16
x = 3 -> 2^3 = 8 -> b)

14. Suponha que se pretende negar os 8 bits menos significativos do registo `$3` e colocar a zero os 24 bits restantes. Para obter esse resultado pode ser usada a seguinte sequência de instruções:

a. `addi $1,$0,0x00FF` `nor $3,$3,$1`
b. `addi $1,$0,0x00FF` `xor $3,$3,$1`
c. `addi $1,$0,0x00FF` `and $3,$3,$1`
d. `addi $1,$0,0xFF00` `nor $3,$3,$1`

a)

15. Considere que o segmento de dados de um programa contém as seguintes diretivas:

```
L1: .word  0, 1, 2
L2: .asciiz "NATAL"
    .align 3
L3: .space 20
```

Se o endereço correspondente ao *label* `L1` for `0x00000100`, o endereço a que corresponde o *label* `L3` é:

a. `0x00000118` b. `0x00000114` c. `0x00000108` d. `0x00000110`

l1:4+4+4 = 12
l2: 5+'|0' = 6 -> 12+6 = 0x12
.align 3 -> 0x18 -> a)

16. Em linguagem C, o código que permite obter a soma dos elementos do *array* "a" é:

a.
```
int a[]={1,5};
int s;
int *p;

p=a[0];
s=&p + &(p+1);
```

b.
```
int a[]={1,5};
int s;
int *p;

p=a;
s=p + (p+4);
```

c.
```
int a[]={1,5};
int s;
int *p;

p=a;
s=*p + *(p+1);
```

d.
```
int a[]={1,5};
int s;
int *p;

p=&a;
s=*p + *(p+4);
```

d) ?

17. No formato de vírgula flutuante IEEE 754, precisão dupla, o expoente da quantidade a representar é codificado em:

a. complemento para 2
b. excesso de 1023
c. excesso de 127
d. sinal e módulo

b)

18. No formato de vírgula flutuante IEEE 754, precisão simples, com mantissa normalizada, a menor quantidade positiva representável é codificada como:

a. `0x00C00000`
b. `0x00000001`
c. `0x80000001`
d. `0x00800000`

d)

19. Numa implementação *single cycle* da arquitetura MIPS, o valor corrente do registo PC representa:
   a. o endereço de memória da instrução que está em execução
   b. o endereço de memória da instrução que vai ser executada no ciclo de relógio seguinte
   c. o código-máquina da instrução que está em execução
   d. o endereço de memória onde vai ser escrito o código máquina da instrução que está em execução

a)

20. Na implementação *single cycle* de uma arquitetura usam-se 2 memórias com o objetivo de:
   a. separar os segmentos de dados e de código do programa
   b. melhorar o desempenho da arquitetura, ao permitir a duplicação da frequência de trabalho
   c. permitir o acesso à memória de dados e à memória de instruções no mesmo ciclo de relógio
   d. permitir que o programa em execução possa alterar os valores dos códigos-máquina das instruções

c)

21. Na implementação *multi-cycle* da arquitetura MIPS, na execução da instrução `J`, a unidade de controlo:
   a. ativa o sinal **`PCWrite`** no primeiro e no terceiro ciclo de relógio
   b. ativa o sinal **`MemWrite`** no primeiro ciclo de relógio e o **`PCWrite`** no terceiro ciclo de relógio
   c. ativa o sinal **`PCWriteCond`** no primeiro ciclo de relógio e o **`MemRead`** no terceiro ciclo de relógio
   d. ativa o sinal **`PCWrite`** no primeiro ciclo de relógio e o **`RegWrite`** no terceiro ciclo de relógio

a)

22. O valor disponível na saída do registo **`AluOut`** no quarto ciclo de relógio da execução de uma instrução **`LW`**, numa arquitetura MIPS *multi-cycle* é (**`offset32`** é o *offset* da instrução estendido com sinal para 32 bits):
   a. `PC + 4`
   b. `[rs] + offset32`, em que `[rs]` é o conteúdo do registo codificado no campo `rs` da instrução
   c. `[rt] + offset32`, em que `[rt]` é o conteúdo do registo codificado no campo `rt` da instrução
   d. `PC + (offset32 << 2)`

b)

23. Numa implementação *pipeline* da arquitetura MIPS, os sinais de controlo fundamentais são gerados:
   a. no estágio **`IF`** por um circuito lógico combinatório e propagados para os estágios seguintes a cada transição ativa do sinal de relógio
   b. no estágio **`ID`** por um circuito lógico sequencial, com um número de estados igual ao número de estágios do *pipeline*; os sinais de controlo são função do estado atual da máquina de estados
   c. no estágio **`EX`** por um circuito lógico combinatório e propagados para os estágios seguintes e anteriores, a cada transição ativa do sinal de relógio
   d. no estágio **`ID`** por um circuito lógico combinatório e propagados para os estágios seguintes a cada transição ativa do sinal de relógio

d)

24. Numa arquitetura MIPS *pipelined* a técnica de *forwarding/bypassing* consiste:
   a. na resolução de *hazards* estruturais
   b. na utilização, como operando, de um resultado produzido por outra instrução que se encontra numa etapa mais avançada do *pipeline*
   c. na resolução de *hazards* de controlo
   d. na escrita antecipada do resultado de uma instrução no *Register File*

b)

25. A técnica *delayed-branch* num processador *pipelined* consiste:
   a. na execução da instrução que se encontra a seguir a um *branch* independentemente do resultado *taken* / *not taken* do *branch*
   b. na paragem parcial do *pipeline* atrasando a execução da instrução a seguir a um *branch* até ser conhecido o resultado da instrução de *branch*
   c. no atraso de 1 ciclo de relógio na escrita do *Register File*, nas instruções de *branch*
   d. no atraso de 1 ciclo de relógio na execução da instrução de *branch*

a)

Pergunta bónus (não pode usar dupla)

26. Suponha que se pretende implementar um sistema baseado na arquitetura MIPS *multi-cycle*, dotado de uma memória de 1 kByte organizada em *words* de 32 bits. Os bits do barramento de endereços do CPU que devem ser usados na ligação à memória são:
   a. A_4 a A_0
   b. A_7 a A_2
   c. A_9 a A_2
   d. A_9 a A_0

1 kByte = 1024
log2(1024) = 10
d) ?

Resposta correta: $\begin{cases} bonus = \left(1 - \frac{respostas_{erradas}}{respostas_{certas}}\right) * 0.5 \text{ (totais, incluindo duplas)} \\ = \min(20, nota_{exame} + bonus) \text{ (se bonus} > 0) \end{cases}$

Grupo II

27. Numa implementação *single cycle* da arquitetura MIPS, a frequência máxima de operação, assumindo que os elementos operativos apresentam os atrasos de propagação a seguir indicados, é, aproximadamente:
   Memórias externas: Leitura - 3 ns, Escrita - 2 ns; *Register File*: Leitura – 2 ns, Escrita – 1 ns;
   Unidade de Controlo: 1ns; *ALU (qualquer operação)*: 1 ns; *Somadores*: 1 ns; Outros: 0ns.
   a. 90 MHz
   b. 100 MHz
   c. 125 MHz
   d. 333 MHz

2*3 + 2 + 1 + 1 = 10
1/10 = 100 MHz
b)

28. Suponha que, no *datapath* da Figura 2, imediatamente antes da ocorrência da transição ativa do relógio, os sinais de controlo **`RegDst`**, **`IorD`** e **`MemtoReg`** apresentam os valores lógicos '**`0`**', '**`1`**' e '**`1`**', respetivamente. Pode então concluir-se que:
   a. está em execução uma instrução **`SW`** na fase **`MEM`**
   b. está em execução uma instrução **`LW`** na fase **`MEM`**
   c. não é possível identificar a instrução que está em execução
   d. está em execução uma instrução **`LW`** na fase **`WB`**

d)

29. Pretende-se incluir no *datapath* da Figura 2 suporte para a execução da instrução **`LUI $reg, imm`**. Para além de adaptar a unidade de controlo, é necessário:
   a. acrescentar uma entrada ao *multiplexer* M3 e ligar os 16 bits mais significativos aos 16 bits menos significativos do código máquina da instrução; os restantes 16 bits são colocados a zero
   b. acrescentar uma entrada ao *multiplexer* M3 e ligar os 16 bits mais significativos aos 16 bits mais significativos do código máquina da instrução; os restantes 16 bits são colocados a zero
   c. acrescentar uma entrada ao *multiplexer* M1 e ligar os 16 bits mais significativos aos 16 bits menos significativos do código máquina da instrução; os restantes 16 bits são colocados a zero
   d. acrescentar uma entrada ao *multiplexer* M3 e ligá-la à saída do bloco *sign extend*

a) ?

30. O trecho de código ao lado, a executar sobre uma implementação *pipelined* da arquitetura MIPS, apresenta os seguintes *hazards*:

```
Z: lw   $6,0($2)     #1
   addi $2,$2,4      #2
   add  $5,$5,$6     #3
   slti $1,$6,23     #4
   bne  $1,$5,Z      #5
```

a. um *hazard* estrutural na primeira instrução e *hazards* de dados na primeira, terceira e quinta instruções que podem ser resolvidos com *forwarding* para **EX**

b. um *hazard* de controlo na quinta instrução e *hazards* de dados na terceira, quarta e quinta instruções que podem ser resolvidos com *forwarding* para **EX**

c. um *hazard* de controlo na quinta instrução, um *hazard* de dados na terceira instrução que pode ser resolvido com *forwarding* para **EX** e um *hazard* de dados na quinta instrução que não pode ser resolvido com *forwarding* para **EX**

d. um *hazard* de controlo na quinta instrução e *hazards* de dados na terceira e quarta instruções que podem ser resolvidos com *forwarding* para **EX** e um *hazard* de dados na quinta instrução que pode ser resolvido com *forwarding* para **ID**

d)

31. A representação da quantidade $-40.625_{10} \times 10^{-2}$ no formato IEEE 754 precisão simples, é:

a. **0xBEE80000**
b. **0xBED00000**
c. **0xFF500000**
d. **0xC0D00000**

0,40625 = 0,01101 = 1,101 * 2^(-2)
sinal = 1 (-)
exp = 125 - 127 = -2
1|011 1110 1|101 ...
B E D -> b)

32. Com **$f2=0x40DA0000** e **$f6=0x3E000000**, o resultado da instrução "**div.s $f4,$f2,$f6**" é:

a. **$f4=0x426D0000**
b. **$f4=0x3F5A0000**
c. **$f4=0x02DA0000**
d. **$f4=0x425A0000**

c) ou a)

33. Para implementação num microcontrolador, pretende-se definir um novo formato de vírgula flutuante de 20 bits (baseado no formato IEEE 754), em que se requer uma precisão de 4 casas decimais, maximizando a gama de representação (note que $\log_{10}(2) \equiv 0.30$). Nesse formato, o expoente será codificado em:

a. excesso de 15
b. excesso de 31
c. excesso de 63
d. excesso de 127

n sei

Nas questões deste grupo, tome como referência as tabelas a seguir apresentadas. Admita que o valor presente no registo **PC** corresponde ao endereço da primeira instrução do programa. Considere ainda a implementação *pipelined* da arquitetura MIPS que estudou nas aulas, com ***delayed-branch*** e ***forwarding*** para **EX** e para **ID**.

Address	Value
0x7A44	0x09ABC869
0x7A40	0x012347CD
0x7A3C	0x03D51EAE
0x7A38	0x0F193AB8
0x7A34	0x0B506C9A
0x7A30	0x03C12975

CPU

$2 0x00000081

$6 0x00000000

...

PC 0x00006024

Figura 1

```
L0: addi $5,$0,0x7A34
L1: lw   $3,0($5)
    add  $6,$6,$3
    and  $3,$3,$2
    beq  $3,$2,L1
    addi $5,$5,4
    and  $6,$6,$2
L2: sw   $6,0($5)
```

34. A execução completa do trecho de código fornecido, desde o *instruction fetch* da instrução referenciada pelo *label* **L0** até à conclusão da instrução referenciada pelo *label* **L2**, demora:
    a. 28 ciclos de relógio
    b. 32 ciclos de relógio
    c. 35 ciclos de relógio
    d. 40 ciclos de relógio

d)

35. Quando a instrução "**add $6,$6,$3**" está na sua fase **WB** de execução, o valor do registo PC é:
    a. **0x0000603C**
    b. **0x00006038**
    c. **0x00006034**
    d. **0x00006030**

b) ?

36. Admita que no instante zero, correspondente a uma transição ativa do sinal de relógio, vai iniciar-se o *instruction fetch* da primeira instrução. O valor à saída da ALU na conclusão do décimo ciclo de relógio, contado a partir do instante zero, é:
    a. **0x0B506C9A**
    b. **0x00000080**
    c. **0xFFFFFFFF**
    d. **0x00007A38**

d)